

9. Übungsblatt zur Vorlesung Computergraphik im WS 2024/25

Besprechung am Montag, Dienstag und Mittwoch, 26.01. - 28.01.2026

Aufgabe 1 *Transformationshierarchie und Non-Interleaved-Vertex-Buffers* [4 Votierpunkte]

In dieser Aufgabe wird eine komplexere Szene aus einer glTF-Datei geladen und mittels OpenGL gerendert. Wenn Sie das Projekt öffnen und ausführen, öffnet sich ein Fenster, in dem aber noch nichts gezeichnet wird.

Hinweis: Wir empfehlen das Bauen im **Release** Modus, sofern kein Debugger verwendet wird. Das Laden der Szenendatei dauert im **Debug** Modus deutlich länger.

1. Szenehierarchie und Transformationsmatrizen [3 Votierpunkte]

In `main.cpp` finden Sie die Funktion `addGltfNode`. Dort wird bereits für jeden Knoten innerhalb der glTF-Szene eine Transformationskomponente erzeugt, unserer eigenen Szenenrepräsentation hinzugefügt und die lokale Position, Orientierung und Skalierung gesetzt. Die Transformationskomponente (`TransformComponent`) und unsere Szenenrepräsentation (`Scene`) sind in `scene.h` definiert und ausführlich kommentiert. Alle Transformationskomponenten werden in einem 1D-Array gespeichert (konkret `std::vector<TransformComponent> transform_data`) und ein Szenengraph wird über Index-Referenzen auf diesen Array umgesetzt: Jede Transformationskomponente besitzt hierzu eine Index-Referenz auf ihren Elternknoten, den ersten Kindknoten, sowie den nächsten Geschwisterknoten. Hat ein Knoten jeweils keinen Eltern-, Kind- oder Geschwisterknoten, ist die Index-Referenz jeweils auf `NONE_INDEX` gesetzt.

Für die neu hinzugefügte Transformationskomponente in `scene.transform_data[transform_idx]` erhalten Sie die Informationen zu einem möglichen Elternknoten über den Funktionsparameter `parent_transform_idx`. Setzen Sie alle notwendigen Index-Referenzen, um die Szenenhierarchie korrekt zu übernehmen. Vergessen Sie dafür nicht gegebenenfalls auch die notwendigen Einträge zu Kind- und Geschwisterknoten in bereits existierenden Transformationskomponenten innerhalb von `scene.transform_data` zu setzen.

Berechnen Sie anschließend die Transformationsmatrix von Modell- in Weltkoordinaten (`world_transform`) für `scene.transform_data[transform_idx]` entsprechend. Berücksichtigen Sie dabei, dass `position`, `orientation` und `scale` in `TransformComponent` lokal gegeben sind, d.h. relativ zum Elternknoten, sofern es einen gibt.

Hinweis: `TransformComponent` verwenden eine Quaternion zum Speichern der Orientierung. Die Funktion `glm::toMat4` können Sie verwenden, um daraus eine Rotationsmatrix zu erzeugen.

2. OpenGL Vertex Data [1 Votierpunkt]

In der Funktion `CreateMesh` werden für übergebene Index- und Vertexdaten eines Szenenobjekts die entsprechenden OpenGL-Objekte initialisiert. Für Indexdaten ist dies bereits

implementiert und ein *Vertex Array Object* wird ebenfalls bereits initialisiert. Ergänzen Sie die Initialisierung aller *Vertex Buffer* für die übergebenen Vertexdaten. In Gegensatz zum vorherigen Aufgabenblatt liegen die Vertexdaten jetzt non-interleaved vor! Für jeden Eintrag im Parameter `std::vector<VertexBufferDescriptor> const& vertices`, müssen Sie einen neuen Vertex-Buffer verwenden, der jeweils nur ein einziges Attribut beinhaltet. Sie können sich an Folie 26 aus Vorlesungskapitel 5 orientieren. Details zu den OpenGL API-Funktionen können Sie in den OpenGL Reference Pages nachschlagen.



Abbildung 1: Ausgabe nach korrekter Lösung von Aufgabe 1

Hinweis: Mit den Tasten *W*, *A*, *S*, *D*, *Shift* und *Space* kann man sich im Raum bewegen. Die Pfeiltasten erlauben eine Änderung der Blickrichtung. Alternativ kann auch ein Gamepad verwendet werden (z.B. ein Xbox One oder PS4-Controller).

Aufgabe 2 *Shading* [6 Votierpunkte]

Die Szene soll mit dem Phong-Beleuchtungsmodell beleuchtet werden. Dazu werden im Fragment-Shader die Farbe und Intensität jedes Fragments berechnet. Neben der eigentlichen Beleuchtungsberechnung müssen wir dazu als Vorarbeit die Normale und die Blickrichtung bestimmen. Des weiteren werden in dieser Szene mehrere Lichtquellen verwendet und als letzter Punkt wird Normal-Mapping hinzugefügt.

1. **Normal-Matrix** [1 Votierpunkt]

Die Normale im Vertex-Shader `in_normal` ist in Modell-Koordinaten gespeichert. Da die Beleuchtungsberechnung im Fragment-Shader in Weltkoordinaten stattfindet, ist es notwendig, die Normale entsprechend zu transformieren. Führen Sie im Vertex-Shader die korrekte Transformation mit Hilfe der sogenannten *normal matrix* (eng.) durch.

2. **View-Direction** [1 Votierpunkt]

Berechnen Sie im Fragment-Shader den Vektor in Kamerarichtung. Hierfür ist die Verwendung der View-Matrix notwendig.

3. **Lighting** [2 Votierpunkte]

Führen Sie im Fragment-Shader die Berechnung des reflektierten Lichts für jede der 4 vorhandenen Lichtquellen durch und addieren sie das Ergebnis in der Variable `reflected_light` auf. Informationen über die Lichtquellen sind in den Arrays `per_frame.light_position[i]`

und `per_frame.light_color_intensity[i]` gespeichert. Die Funktion `phongLighting()` ist bereits implementiert und kann verwendet werden. `light_color_intensity[i].rgb` enthält die Farbe der Lichtquelle und der Alphakanal `light_color_intensity[i].a` enthält die Intensität der Lichtquelle. Bei der Berechnung sollte auch berücksichtigt werden, dass die Intensität des einfallenden Lichts mit dem Abstand zur Lichtquelle abnimmt.

Hinweis: In moderner Computergraphik werden anstelle des Phong-Beleuchtungsmodells sogenannte PBR-Beleuchtungsmodelle mit PBR-Materialien (PBR = *Physically Based Rendering*) verwendet. Mit solchen Modellen und Materialien ist eine deutlich realistischere Beleuchtung möglich, jedoch sind die verwendeten Materialparameter (u.a. Albedo, Roughness, Metallic) nicht mit dem Phong-Modell kompatibel. Die Objekte in der geladenen Szene verwenden ein solches PBR-Material, jedoch ist im Fragment-Shader bereits eine Umrechnung auf Phong-Parameter enthalten, sodass wir hier weiterhin das Ihnen bekannte Phong-Modell verwenden können.

4. **Tangent-Space-Matrix** [1 Votierpunkt]

Im nächsten Schritt werden wir Normal-Mapping verwenden. Die Normalenvektoren in einer Normal-Map sind in häufig in einem Koordinatensystem gegeben, das durch die Per-Vertex-Normale, die Per-Vertex-Tangente und eine zu diesen beiden Vektoren orthogonale Bitangente definiert ist, dem sogenannten *Tangent-Space*. Um die Normal-Map zu verwenden, benötigen wir eine Transformationsmatrix, die vom Tangent-Space in Weltkoordinaten abbildet. Diese Matrix lässt sich aus den gegebenen Basisvektoren (Normale, Tangente, Bitangente) des Tangent-Space konstruieren. Berechnen Sie die Matrix im Vertex-Shader.

Hinweis: Der vierte Eintrag der Per-Vertex-Tangente enthält das Vorzeichen der Bitangente für den Tangent-Space.

5. **Normal-Map** [1 Votierpunkt]

Anstelle der bisher verwendeten Per-Vertex-Normale, soll jetzt die Normale für jedes Fragment in einer Textur nachgeschlagen werden, sogenanntes Normal-Mapping. Der Wert aus der Textur wurde bereits in `normal_rgba` gespeichert. Zum Speichern in einer Normal-Map werden Normalenvektoren häufig auf den Bereich $[0, 1]$ abgebildet. Sie müssen den Wert daher zunächst wieder auf den Bereich $[-1, 1]$ abbilden. Berechnen sie anschließend die korrekte Normale für jedes Fragment mit Hilfe des Wertes aus der Normal-Map und der Tangent-Space-Matrix.



Abbildung 2: Ausgabe nach korrekter Lösung von Aufgabe 2

Quellen: Die Modelle aus der Szene stammen von PolyHaven <https://polyhaven.com/>, CC0.